

Software Assignment

Tejas Uppala

AI25BTECH11038

1 AIM:

Implement **image compression using truncated SVD**.

I have gone for the Python + C hybrid program for **RGB** image compression.

2 What I have learnt from the lecture on SVD by Gilbert Strang:

From what I have learnt, Singular Value Decomposition (SVD) is just another method to factorize a matrix. But the specialty of SVD is that it is applicable to any matrix, but this is not the case for other factorization methods, where the matrix was positive semi-definite.

The SVD decomposes a matrix ($\mathbf{A}$) into three key components: two orthogonal matrices ($\mathbf{U}$) and ($\mathbf{V}$), and a diagonal matrix ($\mathbf{\Sigma}$) containing singular values of the matrix ($\mathbf{A}^T \cdot \mathbf{A}$).

The way I think about it is that SVD finds a new set of coordinate directions for the matrix. These directions are chosen so that when the matrix $\mathbf{A}$ acts on a vector in one of these directions, it simply stretches or shrinks the vector by some amount (the corresponding singular value). So instead of doing something complicated, after the right change of basis, the matrix behaves like a simple scaling operation.

The right singular vectors ($\mathbf{V}$) are eigenvectors of ($\mathbf{A}^T \cdot \mathbf{A}$), and the left singular vectors ($\mathbf{U}$) are eigenvectors of ($\mathbf{A} \cdot \mathbf{A}^T$), with singular values being the positive square roots of the eigenvalues of these matrices.

3 PRACTICAL USAGE and ALGORITHM Explanation:

SVD can be used to compress an image, where the image is a matrix of all the pixels in the image.

But practically, finding eigenvalues and vectors is not an efficient way to perform such a regular task. Instead, there are many other algorithms that can perform SVD.

I have used one of those algorithms, namely "**POWER BLOCK ITERATION**", which is just the upgraded version of the Power Iteration Algorithm. My program also uses QR-MGS method which supports the algorithm and will be explained later.

"Block Power Iteration is an extension of power iteration that computes multiple dominant eigenvectors simultaneously by iteratively multiplying a matrix with a block of vectors and re-orthonormalizing the result at each step, typically using QR decomposition to maintain numerical stability".

We repeatedly apply $\mathbf{A}^T \cdot \mathbf{A}$ to the *block* of vectors $\mathbf{Q}$ and re-orthonormalise it (this is where QR comes into play) until the vectors equation with the top k eigenvalues of the matrix.

3.1 MATHS INVOLVED IN THE ALGORITHM:

We are given a matrix $\mathbf{A}$ ($m \times n$)

(1) Low-rank SVD Approximation

$$A \approx \sum_{j=1}^k \sigma_j u_j v_j^T \quad (1)$$

(the matrix multiplication $(U \cdot \Sigma \cdot V^T)$ can be written as given above)

(2) Block Power Iteration

$$Y_t = AV_t \quad (2)$$

$$Z_t = A^T Y_t = A^T A V_t \quad (3)$$

$$V_{t+1} = \text{orth}(Z_t) \quad (4)$$

(3) Modified Gram–Schmidt (QR)

$$v_j \leftarrow v_j - \sum_{k=1}^{j-1} (q_k^T v_j) q_k \quad (5)$$

$$q_j = \frac{v_j}{\|v_j\|} \quad (6)$$

(4) Extracting Singular Triples

$$v_j = V(:, j) \quad (7)$$

$$y_j = Av_j \quad (8)$$

$$\sigma_j = \|y_j\| \quad (9)$$

$$u_j = \frac{y_j}{\sigma_j} \quad (10)$$

(5) Rank-1 Update and Deflation

$$A_{\text{app}} \leftarrow A_{\text{app}} + \sigma_j u_j v_j^T \quad (11)$$

$$A_{\text{res}} \leftarrow A_{\text{res}} - \sigma_j u_j v_j^T \quad (12)$$

EXPLANATION:

The goal is to compute the top k singular values and singular vectors of a matrix A . Instead of computing one singular vector at a time, the block power iteration method works with a *block* of vectors simultaneously. This allows the algorithm to capture multiple dominant singular directions in parallel and improves both numerical stability and convergence speed.

3.2 Relationship to the SVD

Consider the singular value decomposition of A :

$$A = U \Sigma V^T,$$

where U contains the left singular vectors, V contains the right singular vectors, and Σ is a diagonal matrix of singular values $\sigma_1 \geq \sigma_2 \geq \dots$.

Multiplying A by its transpose gives:

$$A^T A = V \Sigma^2 V^T.$$

Thus, $A^T A$ has the same right singular vectors as A , and its eigenvalues are σ_j^2 :

$$A^T A v_j = \sigma_j^2 v_j.$$

PROOF:

$$\mathbf{A}^T \mathbf{A} = V \Sigma^T U^T U \Sigma V^T = V \Sigma^T \Sigma V^T = V \Sigma^2 V^T$$

Since $\Sigma = \Sigma^T$ (diagonal), For a vector v_j ,

$$\mathbf{A}^T \mathbf{A} v_j = V \Sigma^2 V^T v_j$$

Since V^T is an orthonormal matrix, $V^T v_j = e_j$, where e_j is one of the base vectors, From this

$$\mathbf{A}^T \mathbf{A} v_j = V \Sigma^2 e_j = \sigma_j^2 v_j$$

Repeated application of $A^T A$ therefore amplifies components in the direction of the singular vectors with larger singular values.

3.3 Block Iteration

Instead of evolving a single vector, we evolve a block of vectors V :

$$V \leftarrow A^T A V.$$

With each multiplication, components corresponding to larger singular values grow faster, causing the block to converge to the subspace spanned by the top b right singular vectors.

3.4 Orthonormalization via QR

Without normalization, the vectors in the block would collapse toward the strongest singular direction. To prevent this, a QR factorization is applied after each iteration:

$$V \leftarrow \text{qr}(A^T A V).$$

The QR step preserves orthonormality and ensures that the block spans a well-conditioned basis that captures multiple dominant directions simultaneously.

3.5 Recovering Left Singular Vectors

Once the right singular vectors v_j are obtained, the corresponding left singular vectors are computed as:

$$u_j = \frac{Av_j}{\sigma_j},$$

which follows directly from the relation $Av_j = \sigma_j u_j$.

3.6 Deflation

To compute singular vectors beyond the first block, the contribution of the previously computed singular pairs is removed:

$$A \leftarrow A - \sum_{j=1}^b \sigma_j u_j v_j^T.$$

This *deflation* step ensures that subsequent iterations reveal the next dominant singular values instead of converging back to those already computed.

3.7 Application to Color Images

For RGB images, the matrix A differs for each color channel. Therefore, the block power iteration process, including deflation, is applied separately to the Red, Green, and Blue channels. The three reconstructed channels are then stacked to obtain the final compressed image.

3.8 PSEUDOCODE:

Algorithm 1 Block Power Iteration + QR SVD Approximation

Require: $A \in \mathbb{R}^{m \times n}$, target rank k , block size b

Ensure: Approximate SVD: $\{(\sigma_j, u_j, v_j)\}_{j=1}^k$

```

1:  $A_{\text{res}} \leftarrow A$  ▷ Residual matrix
2:  $A_{\text{app}} \leftarrow 0$  ▷ Reconstructed approximation
3:  $\text{PrevV} \leftarrow \text{empty matrix}$  ▷ Stored right singular vectors
4:  $\text{remaining} \leftarrow k$ 
5: while  $\text{remaining} > 0$  do
6:    $\text{curr\_b} \leftarrow \min(b, \text{remaining})$ 
7:   Initialize  $V \in \mathbb{R}^{n \times \text{curr\_b}}$  (random or heuristic)
8:    $V \leftarrow V - \text{PrevV}(\text{PrevV}^T V)$  ▷ Orthogonalize against previous
9:    $V \leftarrow \text{orth}(V)$  ▷ Normalize columns
10:  repeat
11:     $Y \leftarrow A_{\text{res}} V$ 
12:     $Z \leftarrow A_{\text{res}}^T Y = A_{\text{res}}^T A_{\text{res}} V$ 
13:     $V \leftarrow \text{orth}(Z)$ 
14:  until converged
15:  for  $j = 1$  to  $\text{curr\_b}$  do
16:     $v_j \leftarrow V(:, j)$ 
17:     $y_j \leftarrow A_{\text{res}} v_j$ 
18:     $\sigma_j \leftarrow \|y_j\|$ 
19:     $u_j \leftarrow y_j / \sigma_j$ 
20:     $A_{\text{app}} \leftarrow A_{\text{app}} + \sigma_j u_j v_j^T$ 
21:     $A_{\text{res}} \leftarrow A_{\text{res}} - \sigma_j u_j v_j^T$ 
22:  end for
23:  Append first  $\text{curr\_b}$  columns of  $V$  to  $\text{PrevV}$ 
24:   $\text{remaining} \leftarrow \text{remaining} - \text{curr\_b}$ 
25: end while
26: return  $\{(\sigma_j, u_j, v_j)\}_{j=1}^k$  and  $A_{\text{app}}$ 

```

4 RECONSTRUCTED IMAGES VIA MY PROGRAM:

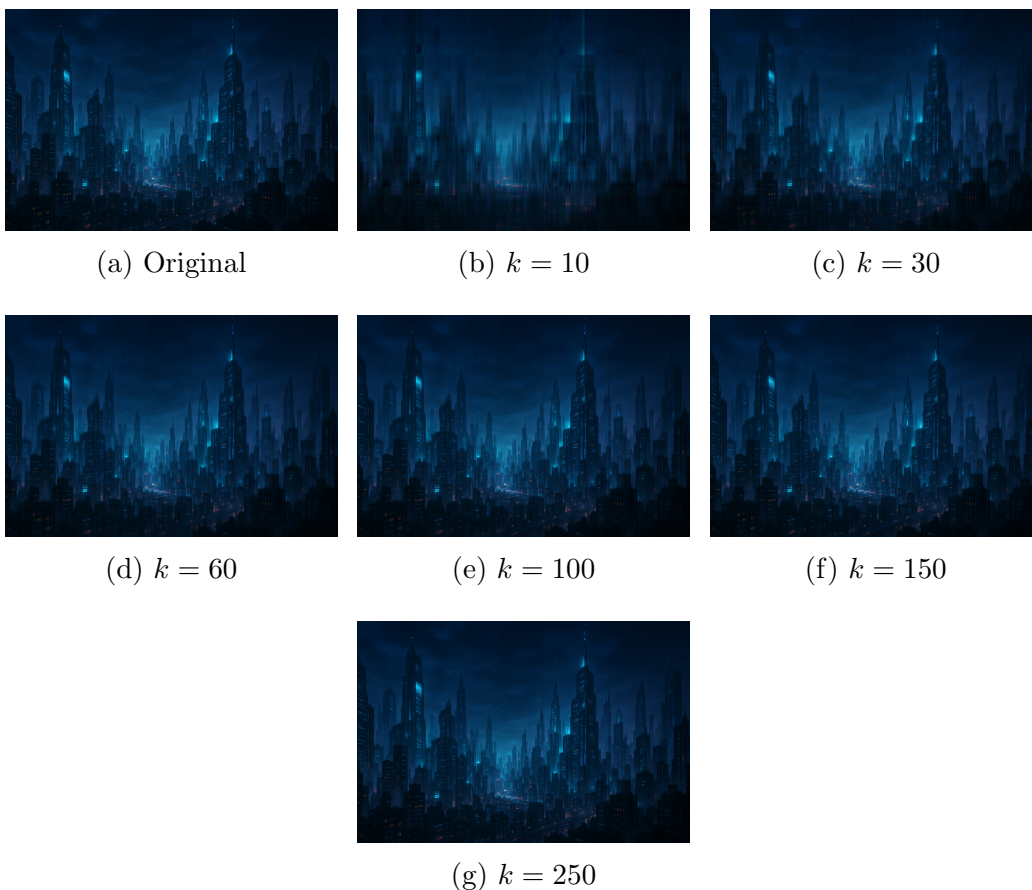


Figure 1: Reconstructed images for varying values of k showing the effect of rank reduction on image quality.

The figures shown above are for this report, the images given in the **google drive folder** are present in the “figs” folder in my repo.

5 ERROR ANALYSIS:

I observed that when I increased k , the reconstructed image improved in its quality. The Frobenius norm measured this quantitatively, and the values matched what I was seeing. So this value gave a meaningful reflection of visual quality in my runs.

5.1 Forbenius Norm:

$$||A - A_k||_F = \sqrt{\sum (A_{ij} - A_{k,ij})^2}$$

We use the Frobenius norm because it matches with what we are trying to measure in SVD image compression, “How much total information of the matrix is lost when we approximate it with rank k .”

The measure of error is usually the ratio of the forbenius norm of $A - A_k$ and A .

→ The lesser the ratio the lesser is the compression.

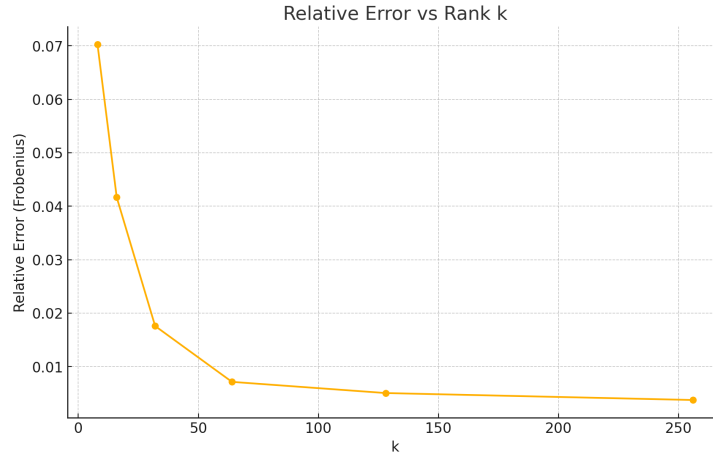


Figure 2: Error Plot

The norm was calculated using the code `forbenius.py` in the `codes` folder, which uses the `numpy.linalg` function `np.linalg.norm()`. The table of various errors is present in the “tables” folder.

5.2 Trade-off Between Rank k , Image Quality, and Compression

The choice of k directly controls the balance between compression efficiency and image quality. A low value of k means that only the most significant singular values and vectors are retained, which results in a highly compressed representation. However, since the unused singular values also contain visual detail, using a very small k can cause blurring.

As k increases, more singular components are included in the reconstruction, so image quality improves. The Frobenius norm error and the relative error curves confirm this behavior: the error drops rapidly when k increases from 8 to 32, meaning the most important structure of the image is captured early. Beyond $k \approx 64$, the improvement in quality becomes minimal, demonstrating diminishing returns. At the same time, storage cost and computation increase linearly with k .

(I have taken only multiples of 16 as my block size is of 16 vectors. I chose a block size of 16 mainly because it gave a good balance between speed and accuracy during experimentation. When I tried smaller blocks, convergence was slower and sometimes unstable. Larger blocks worked but increased memory usage and made the code run slower)

6 WHY BLOCK POWER ITERATION:

Power Iteration was probably one of the first algorithms that I found on the internet for finding the dominant eigenvector. The idea is straightforward: keep multiplying a vector by the matrix, and eventually the direction of that vector aligns with the most dominant singular direction. The limitation is that this method only gives one vector at a time. So if we need a rank- k approximation, running standard power iteration repeatedly becomes slow and also numerically unstable.

(It took me 80 mins to run a $k = 20$ iteration!)

Using the block version allowed me to compute the top k right singular vectors together, instead of one-by-one. In practice, this made the implementation faster. I found it easier to code in C using only the limited libraries compared to more complex SVD algorithms.

There are more advanced algos like Lanczos or Randomized SVD, which are known to be faster in very large-scale problems. However, for this project,

Block Power Iteration felt appropriate.

7 STRUCTURE AND EXPLANATION OF PROGRAM:

- The python code, “**dismantle.py**” reads the image and converts it into a numpy array (3D for RGB), and creates five files, **k.txt** for the value of k; [**a.bin**, **b.bin**, **c.bin**] as the three 2D matrices for the three channels R,G and B respectively.and **shape.txt** for the dimensions of these matrices.
- The C code, “**block_power.c**” reads all of the above files and runs the algo and outputs 3 files, [**a_out.bin**, **b_out.bin**, **c_out.bin**] as the k - truncated 2D matrices.
- The python code, “**reconstruction.py**”, reconunstructs the images form the matrices in the binary files.